

Java web 技术栈

B/S (Browser/Server) 浏览器/服务器 架构

特点: 客户端只需要浏览器, 应用程序的逻辑和数据都存储在服务器端,

浏览器只需请求服务器, 获取Web资源. 服务器把Web资源发送给浏览器.

好处: 易于维护升级, 服务器端升级后, 客户端无需部署即可使用新版本



HTTP (HyperText Transfer Protocol) 超文本传输协议 → 请求数据分为

- 请求行
- 请求头 3 部分
- 请求体

规定了浏览器与服务器之间的数据传输规则

HTTP 协议的特点:

- 基于TCP协议, 面向连接, 安全
- 基于“请求-响应”模型, 一次请求对应一次响应
- HTTP 协议是无状态的协议, 对事务处理没有记忆能力, 每次请求-响应都是独立的

响应数据分为

- 响应行
- 响应头
- 响应体 3 部分

↓

状态码:

- 1xx 响应中
- 2xx 成功
- 3xx 重定向
- 4xx 客户端错误
- 5xx 服务器错误

优点: 速度快

缺点: 多次请求间不能共享数据, Java 中使用会话技术(Cookie, Session) 解决

GET 请求与 POST 请求的区别:

{ GET 请求的参数会暴露在 URL 里
POST 请求的参数藏在请求体里

① GET 请求的请求参数在请求行中, 没有请求体;

POST 请求的请求参数在请求体中

② GET 请求的请求参数大小有限制;

POST 请求的请求参数没有大小限制

Get 方法

从 Web 服务器获取信息 (如文档, 图表, 数据库查询结果等)

当在浏览器中直接输入 URL 或单击 hyperlink 时, 就向 Web 服务器发出了一个 Get 请求

Get 请求可以在 URL 后面添加一些参数发送给 Web 服务器

例如 `http://localhost/getuserinfo.html?id=100&name=abc`

Get 方法可用于提交 HTML 表单 (form)

Post 方法

向 Web 服务器发送信息

一般和 HTML 的表单标记使用, 用于录入数据并传递给 Web 服务器

Get 与 Post 的区别:

- ① Get 提交的数据会放在 URL 之后, 用 ? 分割 URL 和传输数据, 参数之间以 & 相连;
Post 把提交的数据放在 HTTP 包的 Body 中。
- ② Get 方式明文传递, 数据量小, 不安全;
Post 方式密文传递, 数据量大, 安全。
- ③ Get 是浏览器默认的请求方式, 效率高;
Post 没有 Get 效率高。
- ④ Get 对应的 Servlet 方法是 doGet;
Post 对应的 Servlet 方法是 doPost。

Servlet

用Java语言编写的服务器端小程序,驻留在web服务器并在其中运行,扩展了web服务器的动态处理功能

Servlet是一个接口,定义了Java类被浏览器访问到(被tomcat识别到)的规则

我需要定义一个类,实现Servlet接口,复写(override)其中的方法

创建servlet有2种方法:

- ① 通过继承HttpServlet抽象类并覆盖其中部分方法(如doGet(), doPost()),
在web.xml文档中配置<servlet-mapping></servlet-mapping>
- ② 直接new servlet, 通过@WebServlet("/ServletDemo2")注解配置(servlet 3.0以上支持)

Servlet的生命周期

生命周期方法:

Servlet是单例的,在内存中只存在一个对象
存在多用户访问时的线程安全问题,故尽量避免在Servlet中定义成员变量,
即使定义了也不要修改该变量的值

init(): 初始化方法,在servlet被创建时执行,只会执行一次

service(): 提供服务的方法,当浏览器使用get方式提交数据时,service提供doGet方法,
当浏览器使用post方式提交数据时,service提供doPost方法

每次servlet被访问时执行

destroy(): 销毁方法, tomcat正常关闭时,执行一次

Servlet对象默认第一次访问时被创建

也可以指定:

要使之第一次访问时被创建,则设<load-on-startup>值为负

要使之在服务器启动时被创建,则设<load-on-startup>值为0或正

Servlet实例化之后接收客户端请求并作出响应都是通过调用service()方法来实现的

用户在实现具体的Servlet时,一般不重载service方法,web容器在调用service()时,
会依据请求方式的不同自动调用doGet(), doPost(), doPut(), doDelete中的一种或几种,

只需重载对应的doXxx()即可。

例如:

```
public void doGet(HttpServletRequest req, HttpServletResponse resp) {  
    ...  
}  
  
public void doPost(HttpServletRequest req, HttpServletResponse resp) {  
    ...  
}
```

servlet 中的请求和响应

javax.servlet.http 包的接口:

HttpServletRequest 接口 提供 HTTP 请求信息

HttpServletResponse 接口 提供 HTTP 响应

HttpServletRequest 接口的部分常用方法

后端获取参数值是根据 name 获取的

`request.getParameter(参数1)`

获取请求中的单个参数值并返回字符串

假设前端表单有: `<input name="username" value="张三">`, 后端获取: `String username = request.getParameter("username");`

... "张三"

`request.getParameterValues(参数1)`

获取请求中同一个 key 对应的多个值并返回字符串数组

假设前端表单有: `<input type="checkbox" name="hobby" value="风"><input type="checkbox" name="hobby" value="舞">`

后端获取: `String[] hobbies = request.getParameterValues("hobby");` ... `["风", "舞"]`

`request.setCharacterEncoding(参数1)` ... 仅对 Post 请求有效

设置请求参数的字符编码, 解决中文乱码问题

`request.setCharacterEncoding("UTF-8");` ... `getParameter` 获取的中文不会乱码

往 request 域中
临时存取
数据(仅在
一次生命周期
内有效)

`request.setAttribute(参数1, 参数2)`

`request.setAttribute("username", "张三");`

`request.setAttribute("age", "20");`

`request.getAttribute(参数1)` ... 返回 object 类型, 需强转!

`String uname = (String) request.getAttribute("username");`

`Integer age = (Integer) request.getAttribute("age");`

`request.getContextPath()`

获取当前 Web 项目的访问路径前缀 (即 项目名)

`String path = request.getContextPath();`

`request.getCookies()`

获取浏览器请求时携带的所有 Cookie 对象, 返回 `Cookie[]` 数组

`Cookie[] cookies = request.getCookies();`

`request.getSession()`

获取当前请求对应的 Session 对象

`HttpSession session = request.getSession();`

例. 用 Servlet 获取表单输入信息

```
<!DOCTYPE html>
<html>
<head>
  <title> doPOST 测试</title>
</head>
<body>
  <h2> 请输入用户信息: </h2>
  <form method="post" action="/servlet/myServlet2">
    <label> 用户名: </label>
    <input type="text" name="username"><br>
    <label> 密码: </label>
    <input type="password" name="password"><br>
    <input type="submit" value="提交">
    <input type="reset" value="重置">
  </form>
</body>
</html>
```

其值可写绝对路径(以"/"开头)或相对路径

@ WebServlet ("/servlet/myServlet2")

```
public class MyServlet2 extends HttpServlet {
```

@Override

```
protected void doPost (HttpServletRequest request, HttpServletResponse response) throws IOException {
```

★ < request.setCharacterEncoding("UTF-8"); 表单以post方式提交到servlet后的中文乱码问题 (get方式不需要)

response.setContentType("text/html; charset=UTF-8"); → 避免响应内容中文乱码

```
PrintWriter out = response.getWriter();
```

```
String username = request.getParameter("username");
```

```
String password = request.getParameter("password");
```

```
out.write("<html>");
```

```
out.write("<head><title> 表单结果</title></head>");
```

```
out.write("<body>");
```

```
out.write("<h3> 您输入的信息是: </h3>");
```

```
out.write(" 用户名: " + username + "<br>");
```

```
out.write(" 密码: " + password + "<br>");
```

```
out.write("</body>");
```

```
out.write("</html>");
```

```
}
```

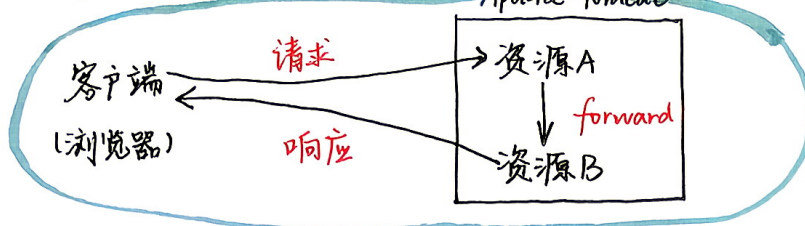
```
}
```

request 请求转发

forward

一种在服务器内部的资源跳转方式

实现方式: `req.getRequestDispatcher("资源路径").forward(req, resp)`



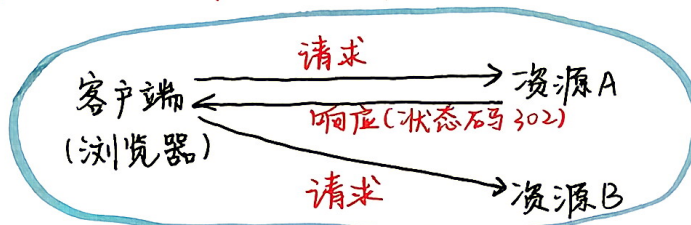
Servlet 之间的通信是通过请求转发来实现的

request 请求转发的特点:

- ① 浏览器地址栏路径不变
- ② 只能转发到当前服务器的内部资源
- ③ 一次请求, 可以在转发的资源间使用 request 共享数据
- ④ 执行速度快, 在 Web 服务器中运行

response 重定向

一种资源跳转方式



实现方式:

`resp.sendRedirect("资源B的路径")`

response 重定向的特点:

- ① 浏览器地址栏路径变化
- ② 可以重定向到任意位置 (服务器内部与外部资源均可)
- ③ 两次请求, 不可以在多个资源间使用 request 对象共享数据
- ④ 执行速度慢, 需要往返客户端

资源路径问题:

request 资源转发不可以加虚拟目录

response 重定向可以加虚拟目录 (项目访问路径)

重定向中虚拟目录的动态获取方法:

```
String contextPath = req.getContextPath();  
resp.sendRedirect(contextPath + "/index.jsp");
```

超链接和表单中也可以用动态获取的方法:

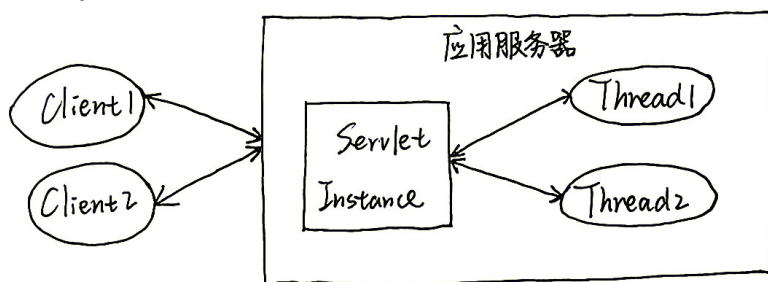
```
<a href=" (%) req.getContextPath(%) /index.jsp" >访问</a>
```

`<% =` `%>` 是 JSP 表达式标签

多线程

Servlet 由应用服务器装入, 同一时间只有一个 Servlet 对象的实例, 并且它在 Servlet 的使用期间将一直保留。

当两个客户同时请求 Servlet 服务, 服务器会根据实例对象为每个请求创建一个处理程序线程, 每个线程处理它自己的请求, 服务器将响应发送回相应的客户机。



多线程可能带来不安全问题。

解决方法:

① 尽量不要使用成员变量或静态成员变量, 即使使用了也不要修改其值

② 添加同步代码块:

```
synchronized(this)
{
    ...
}
```

例. 实现页面登录功能, 用户输入 "admin" "123" 则登录成功, 跳转到欢迎页面, 否则登录失败返回登录页面

(1) 欢迎页面 (success.jsp) 使用 request 请求转发的方式跳转, 并在欢迎页面动态显示用户名信息

(2) 登录页面 (login.jsp) 使用 response 重定向跳转

欢迎您, `<% = request.getAttribute("username") %>`! 登录成功!

JSP 表达式标签 `<% = %>` 和 EL 表达式 `${ }` 会

从 request 域中读取键为 "username" 的值 自动完成类型转换, 但纯 Java 中需要强转

这个值之前 Servlet 通过 `request.setAttribute("username", 用户名);` 存入

```
resp.setContentType("text/html; charset=utf-8");
```

```
req.setCharacterEncoding("UTF-8");
```

```
String username = req.getParameter("username");
```

```
String pwd = req.getParameter("password");
```

```
if ("admin".equals(username) && "123".equals(pwd)) {
```

```
    req.setAttribute("username", username); // 往 request 域中存值
```

```
    req.getRequestDispatcher("/success.jsp").forward(req, resp);
```

```
} else {
```

```
    resp.sendRedirect("/WebShiYan501/login.jsp");
```

```
}
```

例. 实现页面注册功能, 用户点击提交按钮就把表单注册信息提交到后台 regServlet, 由 regServlet 获取用户注册信息并输出到前端页面

doGet:

```
response.setContentType("text/html; charset=utf-8");
```

```
PrintWriter out = response.getWriter();
```

```
String gender = request.getParameter("gender");
```

```
String[] hobbies = request.getParameterValues("hobby");
```

```
String xueli = request.getParameter("xueli");
```

```
String introduction = request.getParameter("introduction");
```

```
String pro = request.getParameter("pro");
```

```
out.println("<h1>表单信息展示</h1>");
```

```
out.println("<p>性别:" + (gender == null ? "未选择" : gender) + "</p>");
```

```
out.println("<p>爱好:" );
```

```
if (hobbies != null) {
```

```
    for (String hobby : hobbies) {  
        out.println(hobby + " ");  
    }
```

```
for (int i = 0; i < hobbies.length; i++) {  
    String hobby = hobbies[i];  
    out.println(hobby + " ");  
}
```

```
} else {
```

```
    out.println("未选择");
```

```
}
```

```
out.println("</p>");
```

```
out.println("<p>");
```

```
out.println("<p>学历:" + xueli + "</p>");
```

```
out.println("<p>自我介绍:" + introduction + "</p>");
```

```
out.println("<p>是否同意协议:");
```

```
if (pro == null) {
```

```
    out.println("未同意");
```

```
} else {
```

```
    out.println("同意");
```

```
}
```

```
out.println("</p>");
```

会话跟踪技术

Web应用程序的请求与响应是基于无状态协议HTTP的,意味着服务器不了解有关同一用户以前请求的信息.而有些功能必须由多次请求来完成,例如购物车. Web程序必须有方式得知用户采购了哪些商品,这种在多次请求间共享数据的方式就是会话跟踪技术

会话: 用户打开浏览器访问服务器资源,会话建立,直至有一方断开连接,会话结束.

一次会话中包括多次请求和响应

会话跟踪是一种维护浏览器状态的方法,服务器需识别多次请求是否来自同一浏览器,以便在同一次会话的多次请求间共享数据

客户端会话跟踪技术: Cookie

服务端会话跟踪技术: Session

Cookie

客户端会话技术, 将数据保存到客户端, 以后每次请求都携带Cookie数据进行访问

发送Cookie

- ① 创建Cookie对象, 设置数据

```
Cookie cookie = new Cookie("key", "value");
```

- ② 发送Cookie到客户端, 使用response对象

```
response.addCookie(cookie);
```

获取Cookie

- ③ 使用request对象获取客户端携带的所有Cookie

```
Cookie[] cookies = request.getCookies();
```

- ④ 遍历数组, 获取每一个Cookie对象

```
for (Cookie cookie : cookies) { ... }
```

- ⑤ 使用Cookie对象方法获取数据

```
cookie.getName();
```

```
cookie.getValue();
```

Cookie 存活时间:

默认情况下, Cookie存在浏览器内存中, 当浏览器关闭, 内存释放, Cookie被销毁

可设置: `setMaxAge(int seconds)`

- 0: 删除对应的Cookie
- 正数: 将Cookie写入浏览器所在电脑的硬盘, 持久化存储, 到时自动删除
- 负数: 默认值, Cookie在当前浏览器内存中, 浏览器关闭时被销毁

Cookie 存储中文:

默认情况下, Cookie不能直接存储中文

要存中文, 则需转码:

⇒ 存Cookie时使用URL编码

```
URLEncoder.encode(value, "utf-8");
```

⇒ 取Cookie时, 使用URL解码

```
URLDecoder.decode(value, "utf-8");
```

```
String value = "张三";
value = URLEncoder.encode(value, "utf-8");
Cookie cookie = new Cookie("username", value);
response.addCookie(cookie);

Cookie[] cookies = request.getCookies();
for (Cookie cookie : cookies) {
    String name = cookie.getName();
    if ("username".equals(name)) {
        String value = cookie.getValue();
        value = URLDecoder.decode(value, "utf-8");
        System.out.println(value);
    }
}
```

例. 使用Cookie实现一周内免登录

Serv2Demo.java:

doGet 逻辑

```
response.setContentType("text/html; charset=utf-8");
```

```
String username = request.getParameter("username");
```

```
String pwd = request.getParameter("password");
```

```
Cookie unameCookie = new Cookie("username", username);
```

```
Cookie pwdCookie = new Cookie("pwd", pwd);
```

```
unameCookie.setMaxAge(7*24*3600);
```

```
pwdCookie.setMaxAge(7*24*3600);
```

```
response.addCookie(unameCookie);
```

```
response.addCookie(pwdCookie);
```

index.jsp:

<%

```
String uname = " ";
```

```
String password = " ";
```

```
Cookie[] cookies = request.getCookies();
```

```
if (cookies != null) {
```

```
    for (Cookie cookie : cookies) {
```

```
        String username = cookie.getName(); 得 "username" ①
```

```
        String pwd = cookie.getName();
```

```
        if ("username".equals(cookie.getName)) {
```

```
            uname = cookie.getValue(); 得 username ②
```

```
        } if ("pwd".equals(pwd)) {
```

```
            password = cookie.getValue();
```

```
        } }
```

```
if ("admin".equals(uname) && "123".equals(password)) {
```

```
    request.setAttribute("uname", uname);
```

```
    request.getRequestDispatcher("/welcome.jsp").forward(request, response);
```

%>

当前保存的用户名: <%= uname %>

welcome.jsp:

<%

```
String username = (String) request.getAttribute("uname");
```

%>

<%= username %> 欢迎您!

doPost 逻辑

```
request.setCharacterEncoding("utf-8");
```

```
doGet(request, response);
```

上面获取的用户输入的用户名作Cookie的值

Cookie的名称

① ②

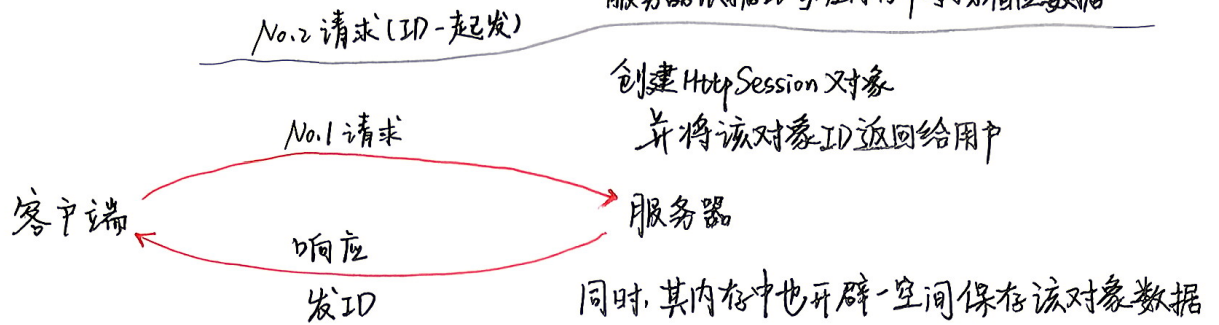
返回 object 类型, 要强转

Session

服务端会话跟踪技术, 将数据保存到服务端

JavaEE 提供了 HttpSession 接口, 实现一次会话的多次请求间数据共享功能

服务器依据 ID 号在内存中寻找相应数据



在一次会话中, 多次获取的 session 对象是同一个对象

获取 Session 对象: 注意类型转换 (默认是 Object)

```
HttpSession session = request.getSession();
```

Session 对象功能:

```
void setAttribute (String name, Object o) // 存数据到 Session 域
```

```
Object getAttribute (String name) // 依据 key 获取值
```

```
void removeAttribute (String name) // 依据 key 删除键值对
```

例如.

```
HttpSession session = request.getSession();
```

```
session.setAttribute ("username", 张三);
```

```
Object name = session.getAttribute ("username");
```

```
System.out.println ("Session 中的用户名: " + name); // 输出到服务器控制台
```

```
response.getWriter().write ("欢迎您: " + name); // 输出到浏览器页面
```

Session 钝化、活化:

(1) 钝化: 服务器正常关闭后, Tomcat 自动将 Session 中的数据存至硬盘文件.

(2) 活化: 再次启动服务器后, 从文件中加载数据到 Session

Session 销毁:

(1) 默认 无操作, 30 分钟自动销毁 (可在 web.xml 的 <session-config> 中配置).

(2) 调用 Session 对象的 invalidate() 方法手动销毁

例. 使用 Session 实现登录页面的验证码功能

(1) 展示验证码图片,  和点击都可以切换

(2) 验证码校验

LoginServ.java:

```
request.setCharacterEncoding("UTF-8");  
response.setContentType("text/html; charset=UTF-8");
```

```
String username = request.getParameter("username");
```

```
String pwd = request.getParameter("password");
```

```
String checkcode = request.getParameter("checkname");
```

```
HttpSession session = request.getSession();
```

```
String code = (String) session.getAttribute("checkcode");
```

```
if (code == null || !code.equalsIgnoreCase(checkcode)) {
```

```
    session.setAttribute("error", "验证码错误!");
```

```
    response.sendRedirect("index.jsp");
```

```
    return;
```

```
}
```

```
if ("admin".equals(username) && "123".equals(pwd)) {
```

```
    session.setAttribute("user", username);
```

```
    response.getWriter().write("登录成功!");
```

```
} else {
```

```
    session.setAttribute("error", "用户名密码错误!");
```

```
    response.sendRedirect("index.jsp");
```

```
}
```

```
或 response.getRequestDispatcher("index.jsp").forward(request, response);
```

chenggong.jsp:

```
<%
```

```
String user = (String) session.getAttribute("user");
```

```
if (user == null) {
```

```
    response.sendRedirect("index.jsp");
```

```
    return;
```

```
}
```

```
%>
```

```
<h1><% = user %>, 登录成功!</h1>
```

index.jsp:

```
<%
```

```
String error = (String) session.getAttribute("error");
```

```
if (error != null) {
```

```
    out.println("<p style='color:red;'>" + error + "</p>");
```

```
    session.removeAttribute("error");
```

```
}
```

```
%>
```

```
<script>
```

```
    const img = document.getElementById('checkImg');
```

```
    img.addEventListener('click', function() {
```

```
        img.src = "CheckServ?" + new Date().getTime();
```

```
    });
```

```
</script>
```

或跳转到
chenggong.jsp

页面输出

Cookie 与 Session 小结

购物车、记住我等应用场景：Cookie

登录后显示用户名、验证码等应用场景：Session

相同点：完成一次会话内多次请求间的数据共享

不同点：

存储位置 - Cookie 将数据存储在客户端，Session 将数据存储在服务端

安全性 - Cookie 不安全，Session 安全

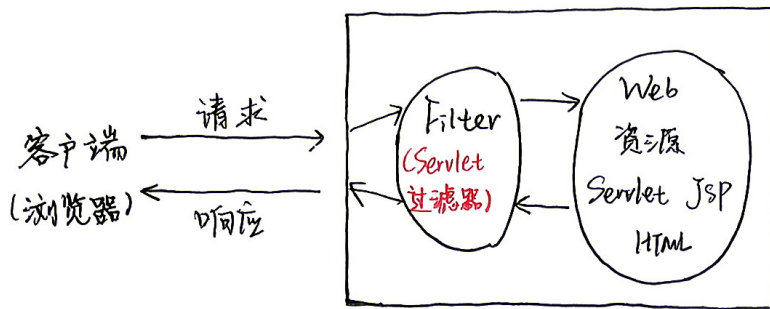
数据大小 - Cookie 最大 3KB，Session 无大小限制

存储时间 - Cookie 可以长期存储，Session 默认 30 分钟

服务器性能 - Cookie 不占服务器资源，Session 占用服务器资源。

Filter 过滤器 (对资源访问进行检查, 符合规则才放行, 否则拦截)

对 Web 服务器管理的 Web 资源进行拦截, 从而实现 URL 级别的权限访问控制, 过滤敏感词汇等



在 Servlet API 中定义了一个接口类 供开发人员编写 Filter 程序 = Filter

Filter 程序是一个实现了 Filter 接口的 Java 类, 和 Servlet 程序一样都由 Servlet 容器调用和执行

Filter 程序需要在 web.xml 文件或注解中进行注册和设置它所能拦截的资源:
JSP, Servlet, 静态图片文件, 静态 html 文件

Filter 可以对客户端的请求和回送给客户端的响应进行拦截,

可以决定是否将请求继续传递给 Servlet 程序, 以及对请求和相应信息是否修改

Filter 执行流程

执行放行前逻辑 → 放行 → 访问资源 → 执行放行后逻辑

对 request 对象中的数据进行处理

对 response 对象中的数据进行处理

一个 Web 应用程序中可以注册多个 Filter 程序, 若有多 Filter 程序对某个 Servlet 程序的访问过程进行拦截, 则 Web 容器把这些 Filter 程序组成 Filter 链

(各 Filter 拦截顺序与其在 web.xml 中的映射顺序一致。

若是注解配置, 则优先级按类名(字符串)自然排序)

Filter 的创建

与开发 Servlet 不同, Filter 接口没有可供继承的实现类, 要开发过滤器只能直接实现 Filter 接口。

① 创建 Filter 接口的实现类并重写其所有方法

```
public class FilterDemo implements Filter {  
    public void init(FilterConfig filterConfig) // 部署初始化参数  
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)  
    public void destroy() {} // Web 容器卸载 Filter 对象前被调用, 在 Filter 生命周期中仅执行一次  
    // 可释放过滤器使用的资源  
}
```

② 在 doFilter 中编写代码

③ 配置 Filter 拦截资源的路径

方法一: @WebFilter("//*")

方法二: <filter-mapping></filter-mapping>

过滤器的拦截路径一般有这 4 种方式:

- ① 拦截所有资源 /*, 访问所有资源, 过滤器都会被执行
- ② 具体资源路径 /mySer/demo1, 仅访问 /mySer/demo1 资源时过滤器才被执行
- ③ 拦截目录 /mySer/*, 访问 /mySer 下所有资源时, 过滤器都会被执行
- ④ 后缀名拦截 *.jsp, 访问后缀名为 jsp 的资源时, 过滤器被执行

doFilter() 方法类似于 Servlet 接口的 service 方法。

当客户端请求目标资源时, 容器调用与这个目标资源相关联的过滤器的 doFilter 方法

参数 request, response 为请求和响应对象。

参数 chain 为当前 Filter 对象, 特定操作完成后可调用当前 Filter 对象的 doFilter 方法请求放行

该方法的请求和响应参数类型分别是 ServletRequest 和 ServletResponse。

即: 过滤器的使用并不依赖于具体协议

典型: 字符编码的过滤器

```
... doFilter(...) {  
    response.setCharacterEncoding("text/html; charset=utf-8");  
    request.setCharacterEncoding("utf-8");  
    chain.doFilter(request, response); // 放行  
}
```

例. 登录验证

LoginFilter /*

判断用户是否登录 { 是: 放行
未登录: 跳转至登录页面并给出提示信息

LoginFilter.java:

@WebFilter ("/*")

public class LoginFilter implements Filter {

@Override

public void doFilter(...) ... {

HttpServletRequest req = (HttpServletRequest) request; // 强转, 否则无法获取Session对象

HttpServletResponse resp = (HttpServletResponse) response;

String[] allowUrls = { "/login.jsp", "/ServDemo4", "/css/", "/images/" }; // 与登录注册有关的无条件放行

String currentUrl = req.getRequestURL().toString(); // 获取当前访问的资源路径

for (String url: allowUrls) {

if (currentUrl.contains(url)) {

chain.doFilter(request, response); // 找到, 无条件放行

return; // 放行后无需其他判断

}

}

判断session中是否有用户

HttpSession session = req.getSession(); // 登录信息

Object uname = session.getAttribute("uname");

if (uname != null) // 已登录

chain.doFilter(request, response); // 放行

} else {

req.setAttribute("login-msg", "您尚未登录"); // 提示信息

req.getRequestDispatcher("/login.jsp").forward(req, resp);

}

}

@Override

... init(...) ... {

}

@Override

... destroy() {

}

}

详见学习通

ServDemo4.java:

doPost:

request.setCharacterEncoding("UTF-8");

response.setContentType("text/html; charset=UTF-8");

String uname = request.getParameter("uname");

String pwd = request.getParameter("pwd");

if ("admin".equals(uname) && "123".equals(pwd)) {

HttpSession session = request.getSession();

session.setAttribute("uname", uname);

response.sendRedirect("success.jsp");

} else {

request.setAttribute("login-msg", "用户名错误!");

request.getRequestDispatcher("/login.jsp").forward(request, response);

login.jsp:

<%

String msg = (String) request.getAttribute("login-msg");

if (msg != null) {

out.println("<p style='color:red;'>" + msg + "</p>");

}

%>

<form action="ServDemo4" method="post">

用户名: <input type="text" name="uname">

密码: <input type="password" name="pwd">

<input type="submit" value="登录">

</form>

JDBC Java 数据库连接

JDBC是使用Java语言操作关系型数据库的一套标准API,

真正执行的代码是各数据库厂商提供的数据库 jar包 中的实现类
放在 WebContent 的 lib 文件夹下

各数据库厂商使用相同的接口, Java代码不需要针对不同的数据库分别开发,
可随时替换底层数据库而访问数据库的Java代码基本不变

操作数据库的方法

① 注册驱动

```
Class.forName("com.mysql.jdbc.Driver");
```

② 获取连接

```
Connection conn = DriverManager.getConnection(url, username, password);
```

③ 定义SQL语句

```
String sql = "update ...";
```

④ 获取执行SQL对象

```
Statement stmt = conn.createStatement();
```

⑤ 执行SQL

```
stmt.executeUpdate(sql);
```

⑥ 处理返回结果

⑦ 释放资源

JDBC的几个API:

① DriverManager

② Connection

③ Statement

④ ResultSet

⑤ PreparedStatement

DriverManager (驱动管理类)

- (1) 注册驱动 `Class.forName("com.mysql.jdbc.Driver");`
- (2) 获得数据库连接 `Static Connection`
`getConnection(String url, String user, String password)`
 连接路径 用户名 密码
 ↓

语法: `jdbc:mysql://ip地址(域名):端口号/数据库名称?参数键值对1&2...`

Connection (数据库连接对象)

- (1) 获取执行sql的对象

• 执行普通的sql对象

// 获取连接

`Connection conn = DriverManager.getConnection(url, username, password);`

// 定义sql语句

`String sql = "update user set password = '111' where username = 'admin'";`

// 获取执行sql语句的Statement对象

`Statement stmt = conn.createStatement();`

- 执行预编译的sql对象 (防止sql注入)

`PreparedStatement preparedStatement(sql);`

- (2) 事务管理

↗ `conn.setAutoCommit(false);`

开启事务: `setAutoCommit(boolean autoCommit)` // true - 自动提交, false - 手动提交

提交事务: `commit()` ↗ `conn.commit();`

回滚事务: `rollback()` ↗ `conn.rollback();`

Statement

执行SQL语句

`int executeUpdate(sql):` 执行DML, DDL语句

返回值: DML语句影响的行数 DDL语句执行后执行成功也可能返回0

`ResultSet executeQuery(sql):` 执行DQL语句

返回值: `ResultSet` 结果集对象

`ResultSet (结果集对象)`

`boolean next():` (1) 将光标从当前位置向前移动一行 (2) 判断当前行是否为有效行 (有数据)

`int/string` ↗ `xxx getXxx (参数):` 获取数据
列编号 列名
(从1开始)

`while (rs.next()) {`

`System.out.println(rs.getString("username"));`

`}`

PreparedStatement

预编译SQL语句并执行, 预防SQL注入问题

通过操作输入来修改事先定义好的SQL语句, 来对服务器进行攻击

① 获取 PreparedStatement 对象

String sql = "select * from user where username=? and password=?";
// SQL语句中的参数值用占位符替代

PreparedStatement pstmt = (conn).prepareStatement(sql);

② 设置参数值

// 通过 Connection 对象获取并传入对应 sql 语句

PreparedStatement 对象: set (数据类型) (参数1, 参数2): 给 ? 赋值

pstmt.setString(1, uname);

pstmt.setString(2, pwd);

? 的位置编号, 从1开始

③ 执行SQL

executeUpdate(); / executeQuery(); // 不必再传递sql

ResultSet rs = pstmt.executeQuery();

例.

用户表 user

管理员账号：增删改查

普通账号：只可修改自己的账号信息

LoginServlet :

doGet :

request.setCharacterEncoding("UTF-8");

response.setContentType("text/html; charset=UTF-8");

PrintWriter out = response.getWriter();

String username = request.getParameter("username");

String password = request.getParameter("password");

try {

Class.forName("com.mysql.jdbc.Driver");

String url = "jdbc:mysql://127.0.0.1:3306/student-db?useSSL=false";

String dbUser = "root";

String dbPwd = "root";

Connection conn = DriverManager.getConnection(url, dbUser, dbPwd);

String sql = "select * from user where username = ' " + username + " ' and password = ' " + password + " ' ";

Statement stmt = conn.createStatement();

ResultSet rs = stmt.executeQuery(sql);

if(rs.next()) {

HttpSession session = request.getSession();

session.setAttribute("username", username);

response.sendRedirect(request.getContextPath() + "/index.jsp");

out.println(username + "欢迎你");

} else {

String errorMsg = "用户名或密码错误";

errorMsg = java.net.URLEncoder.encode(errorMsg, "UTF-8");

response.sendRedirect(request.getContextPath() + "/login.jsp?errorMsg=" + errorMsg);

}

} catch (Exception e) {

e.printStackTrace();

out.println("登录出错!" + e.getMessage());

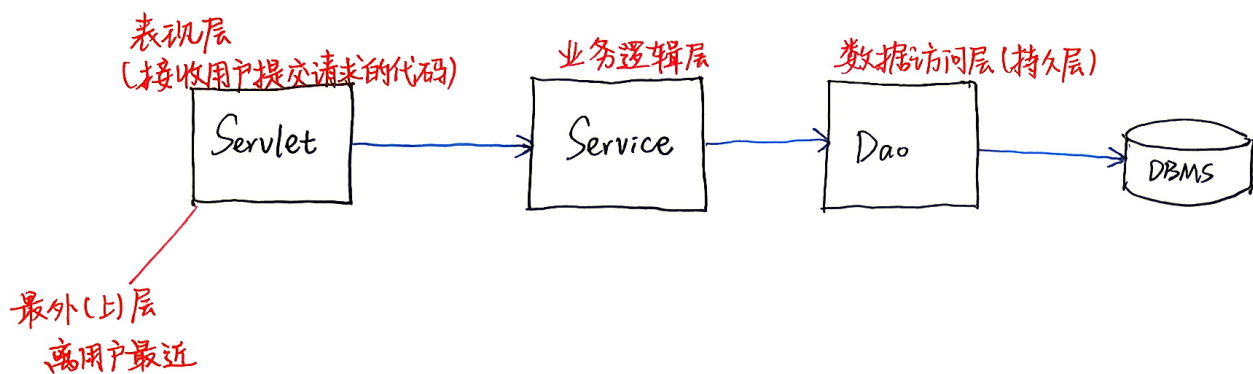
} finally {

try { if(rs != null) rs.close(); if(stmt != null) stmt.close();

if(conn != null) conn.close(); out.close(); } catch (Exception e) {

e.printStackTrace(); }

三层架构



DBUtil 提供通用工具供各层调用

UserBean 数据访问层对象, 封装数据库表数据

采用面向接口编程

上层对下层的调用通过接口来实现。

调用的是接口, 实际提供服务的是接口的实现类

服务标准相同而实现类可以更换, 实现了各层之间的解耦合。

接口